PET & CIA · PROJETO REACT NATIVE

React Native + Expo

Node.js + Express

MySQL

Pet Shop: CRUD com React Native

Gerenciamento de Animais e Serviços

Autora
Ana Sprenger

Curso
Técnico em Informática para Internet · Senac

Data
Outubro de 2026

O que é o projeto

Um app para a recepção de uma pet shop anotar **os animais** e **os serviços** feitos em cada um.

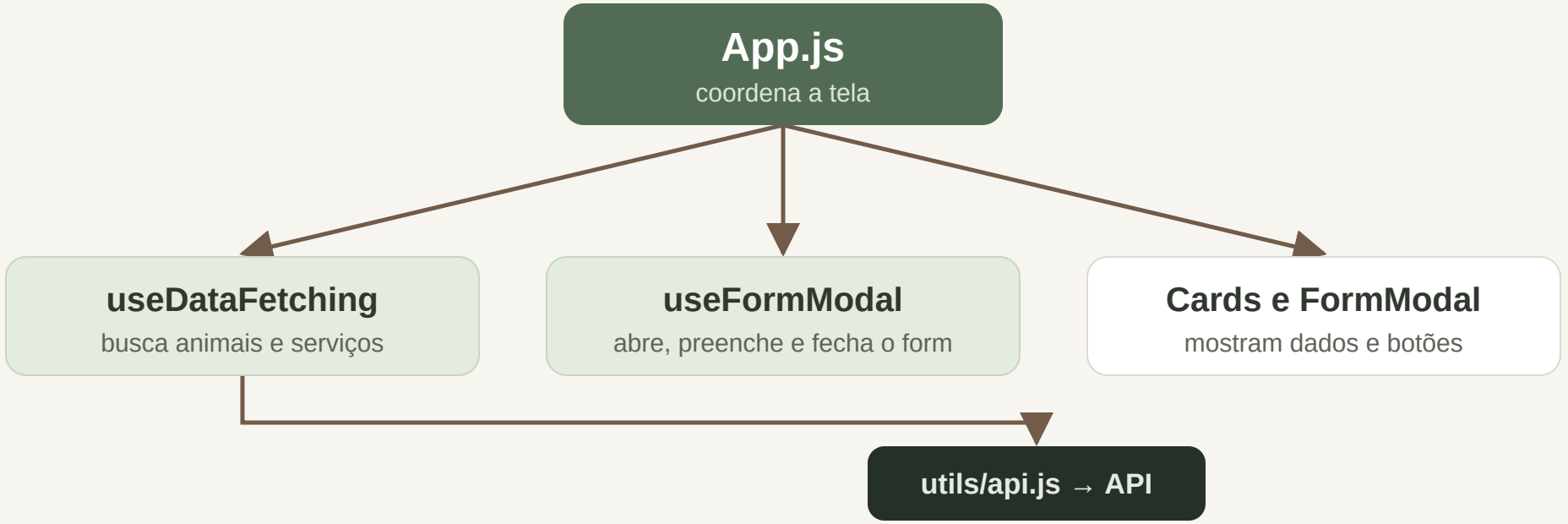
- **App**: React Native com Expo (celular e navegador)
- **API**: Node.js com Express
- **Banco**: MySQL, com duas tabelas



Cada pasta tem uma função

PASTA	PARA QUE SERVE
`App.js`	A tela principal. Junta todas as peças.
`components/`	Peças visuais: os cards e o formulário
`hooks/`	Lógica guardada fora da tela: buscar dados e controlar o formulário
`utils/`	Funções de ajuda: falar com a API e converter datas
`constants/ styles/`	Listas fixas (espécies, status) e as cores do app
`backend/`	A API e a conexão com o MySQL, num projeto separado

Como as peças se conectam



Em resumo: o App.js é o maestro. Os hooks cuidam dos dados, os componentes cuidam do visual.

As 4 operações

POST

Create

NA TELA
Botão + Novo

NO BANCO
`INSERT`

GET

Read

NA TELA
Lista ao abrir o app

NO BANCO
`SELECT`

PUT

Update

NA TELA
Botão Editar

NO BANCO
`UPDATE`

DELETE

Delete

NA TELA
Botão Excluir

NO BANCO
`DELETE`

Padrão: depois de qualquer mudança, o app busca a lista de novo para mostrar o que está no banco.

CRITÉRIO 2 POST CREATE

Cadastrar

1. Tocar em **+ Novo** e preencher o formulário
2. O app confere os campos obrigatórios
3. Envia os dados para a API com **POST**
4. A API grava com **INSERT** e a lista é atualizada

APP.JS · SALVAR()

```
if (!form.nome?.trim()) {
  Alert.alert("Atencao", "Informe o nome...");
  return;
}

await api("/animais", {
  method: "POST",
  body: JSON.stringify(dados),
});
```

CRITÉRIO 2 GET READ

Listar e filtrar

HOOKS/USEDATAFETCHING.JS

```
useEffect(() => {
  carregarDados();   // ao abrir o app
}, []);

const [animaisApi, servicosApi] =
  await Promise.all([
    requisicao("/animais"),
    requisicao("/servicos"),
  ]);
```

- Ao abrir o app, busca animais e serviços **ao mesmo tempo**
- Enquanto espera, mostra "Carregando dados..."
- **Ver serviços** no card filtra a lista daquele animal
- Se a API estiver fora, aparece o botão **Tentar novamente**

CRITÉRIO 2 PUT UPDATE

Editar

Cadastrar e editar usam **o mesmo formulário**. Quem decide é a variável `editando`.

EDITANDO	BOTÃO	REQUISIÇÃO
`null`	Cadastrar	POST /animais
`"1"`	Salvar alteracoes	PUT /animais/1

APP.JS · SALVAR()

```
await api(
  editando
    ? `/animais/${editando}`
    : "/animais",
  {
    method: editando ? "PUT" : "POST",
    body: JSON.stringify(dados),
  }
);
```

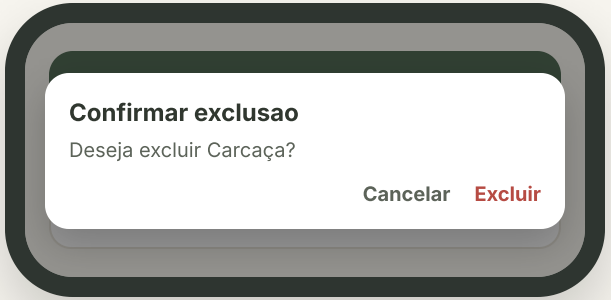
CRITÉRIO 2 DELETE DELETE

Excluir com segurança

1. Tocar em **Excluir**
2. O app pergunta: "Deseja excluir Carcaça?"
3. Só ao confirmar a API recebe o **DELETE**

ON DELETE CASCADE

Ao excluir um animal, o MySQL apaga também os serviços dele. Assim não sobram serviços sem dono.



CRITÉRIO 3 INTEGRAÇÃO

App, API e banco



Comparação: o app é o cliente, a API é o garçom e o banco é a cozinha. O cliente nunca entra na cozinha.

CRITÉRIO 3 INTEGRAÇÃO

Uma requisição, passo a passo

1. O app envia o serviço em JSON
2. A API confere se os dados estão certos
3. A API executa o INSERT no MySQL
4. A API responde **201** e o app atualiza a lista

`POST` `/servicos`

```
{ "animal_id": 2,
  "tipo": "Banho",
  "data": "2026-10-20",
  "valor": 80 }
```

RESPOSTA · 201 CREATED

```
{ "id": 3,
  "mensagem": "Serviço cadastrado com sucesso." }
```

Uma função para falar com a API

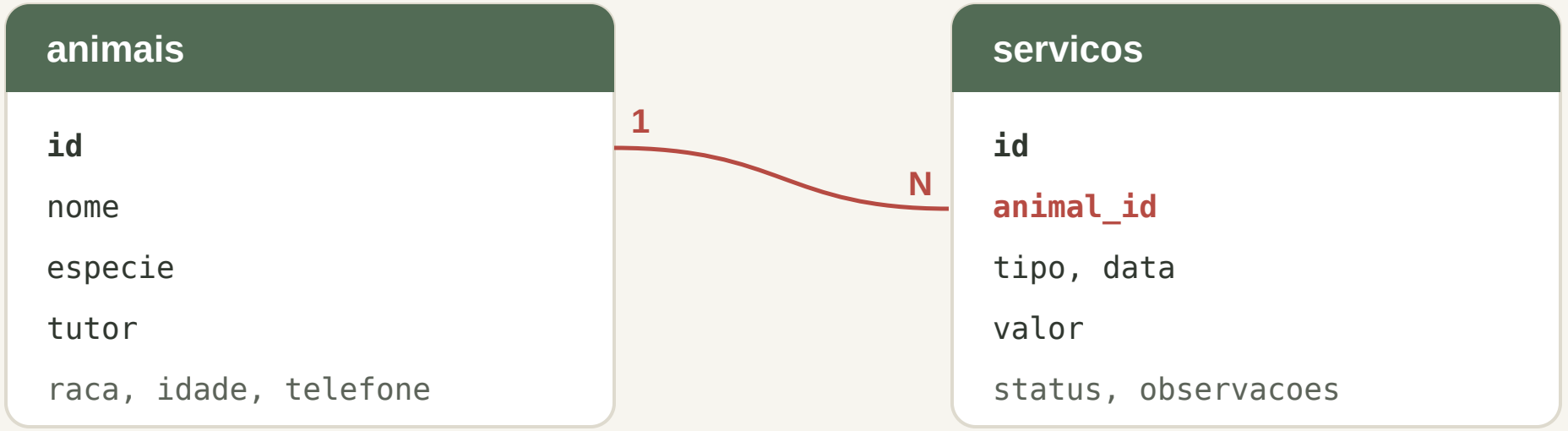
UTILS/API.JS (RESUMIDA)

```
export async function requisicao(caminho, opcoes) {
  const resposta = await fetch(API_URL + caminho, {
    ...opcoes,
    headers: { "Content-Type": "application/json" },
  });
  const dados = await resposta.json();

  if (!resposta.ok) throw new Error(dados.erro);
  return dados;
}
```

- Todo o app usa essa mesma função
- O endereço da API fica em um lugar só
- Se a API responder erro, a mensagem dela vira o aviso na tela

Duas tabelas ligadas



Um para muitos: um animal pode ter vários serviços. O `animal_id` diz de quem é cada serviço.

Como a API recebe um pedido

BACKEND/SERVER.JS

```
app.use(cors());
app.use(express.json());
app.use("/animais", animaisRoutes);
```

BACKEND/ROUTES/ANIMAIS.JS

```
router.post("/", async (req, res) => {
  const { nome, especie, tutor } = req.body;
  if (!nome?.trim()) {
    return res.status(400).json({ erro: "..." });
  }
  const [resultado] = await db.execute(
    "INSERT INTO animais (...) VALUES (?, ?, ?)",
    [nome, especie, tutor]
  );
  res.status(201).json({ id: resultado.insertId });
});
```

- `server.js` liga a API e manda cada endereço para o arquivo de rotas certo
- `req.body` são os dados que o app enviou
- Os `?` no SQL protegem contra SQL Injection
- `res.status()` devolve o código: 201 deu certo, 400 dado errado

Um estado para o formulário inteiro

HOOKS/USEFORMMODAL.JS

```
const [form, setForm] = useState({});

const atualizarCampo = (campo, valor) => {
  setForm((anterior) => ({
    ...anterior,
    [campo]: valor,
  }));
};

// no FormModal, cada campo chama:
atualizarCampo("nome", "Pipoca");
```

- Todos os campos ficam em um único objeto `form`
- `...anterior` copia o que já foi digitado
- `[campo]` troca só a chave que mudou
- Uma função serve para nome, idade, data, valor e todos os outros

Exemplo: `{ nome: "", tutor: "Ana" }` vira `{ nome: "Pipoca", tutor: "Ana" }`

Convertendo dados entre tela e banco

UTILS/DATEFORMATTER.JS

```
export function dataParaBanco(data) {
  // "20/10/2026" → ["20", "10", "2026"]
  const partes = data.match(
    /^(\d{2})\/(\d{2})\/(\d{4})$/
  );
  if (partes) {
    const [, dia, mes, ano] = partes;
    return `${ano}-${mes}-${dia}`;
  }
  return null;  // formato inválido
}
```

APP.JS · VALOR EM REAIS

```
Number(form.valor.replace(",", "."));
```

NA TELA	NO BANCO
20/10/2026	2026-10-20
"80,00"	80.00

- A pessoa digita no formato brasileiro
- O MySQL guarda no formato padrão
- Na volta, `dataParaTela()` faz o caminho contrário

CRITÉRIO 4 DOMÍNIO DO PROJETO

Por que fiz assim

Hooks customizados

Tiram a lógica de dados e do formulário de dentro da tela. O App.js fica só coordenando.

Um só formulário

Cadastro e edição têm os mesmos campos. A variável `editando` evita código repetido.

Validar no app e na API

No app, para avisar rápido. Na API, para proteger o banco mesmo se alguém chamar a API sem o app.

Recarregar a lista após salvar

Garante que a tela mostra o que está no banco, inclusive os serviços apagados pelo CASCADE.

Ideia central: separar responsabilidades. Cada parte do código faz uma coisa só.

CRITÉRIO 4 DOMÍNIO DO PROJETO

O que acontece se...?

...o backend estiver desligado?

O app não trava. Mostra a mensagem de erro e o botão **Tentar novamente**.

...eu excluir um animal com serviços?

O MySQL apaga os serviços junto, por causa do `ON DELETE CASCADE`.

...digitar a data 20-10-2026?

`dataParaBanco()` devolve `null` e aparece "Data invalida".

...tentar cadastrar serviço sem animal?

O botão **+ Novo** fica desativado na aba Serviços enquanto não houver animal.

Por trás disso: a função `requisicao()` transforma qualquer erro da API em uma mensagem na tela.

O que ainda pode melhorar

Limitações hoje

- Não tem login: qualquer um na rede usa a API
- Endereço da API fixo em `localhost`; no celular precisa trocar pelo IP
- A exclusão não avisa que os serviços também somem

Próximos passos

- Login com token (JWT)
- Endereço da API em variável de ambiente
- Avisar quantos serviços serão excluídos
- Busca por nome e filtro por status